



INFORME DEL EXAMEN

I. PORTADA

Tema:	Tema de la guía práctica proporcionada por el docente
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumnos participantes:	Camacho Monta Josue Jampier Chalco Tasna Kenneth Mateo Cunalata Mendoza Damian Alexander Silva Camuendo Luis Alexander Tacuri Santillan Monica Sara Tisalema Guashco Darwin Joel
Asignatura:	Estructura de Datos
Docente:	Ing. José Caiza, Mg.

II. INFORME DEL EXAMEN

2.1 Objetivos

General:

Desarrollar un videojuego de estrategia Tower Defense en Java utilizando estructuras de datos dinámicas y estáticas personalizadas para gestionar eficientemente las entidades y permitir su ejecución en consola e interfaz gráfica (Swing).

Específicos:

- Implementar una lista secuencial manual (ListaSecuencialTorres) para gestionar las estructuras defensivas en el tablero.
- Diseñar una lista doblemente enlazada (ListaDobleEnemigos) para controlar el movimiento, estado y eliminación rápida de enemigos.
- Construir una lista simplemente enlazada circular (ListaCircularOleadas) para coordinar el flujo cíclico de las hordas de ataque.

2.2 Resultados obtenidos

A continuación se documenta el desarrollo del examen: simulador de un videojuego de estrategia Tower Defense en Java, implementado con tres estructuras de datos dinámicas y estáticas construidas manualmente (sin usar ArrayList, LinkedList, Queue, Deque ni ninguna colección de java.util): una lista secuencial para las torres, una lista doblemente enlazada para los enemigos activos y una lista circular simple para el ciclo de oleadas.

1. Ejercicio

Desarrollar un videojuego de estrategia Tower Defense en Java utilizando estructuras de datos dinámicas y estáticas personalizadas para gestionar eficientemente las entidades del juego (torres, enemigos y oleadas) y permitir su ejecución en consola. El jugador registra torres defensivas y oleadas de enemigos, inicia las oleadas y avanza turno a turno: los enemigos se mueven según su velocidad, las torres dañan a los enemigos dentro de su rango, los enemigos destruidos se retiran y cada enemigo que llega al final del camino resta una vida al jugador. La partida termina por derrota (0 vidas) o por victoria (todas las oleadas iniciadas y sin enemigos activos).

2. Solución propuesta

Se organiza el proyecto en clases de datos (Torre, Enemigo, Jugador, Oleada), clases de nodo (NodoEnemigo, NodoOleada), las tres estructuras exigidas (ListaSecuencialTorres, ListaDobleEnemigos, ListaCircularOleadas) y la aplicación de consola con el main.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



Clase	Responsabilidad
Torre	Estructura defensiva: id, nombre, tipo, posición, daño, rango y costo; determina si un enemigo está en rango.
Enemigo	Unidad que avanza por el camino: vida, velocidad, posición y recompensa; recibe daño y se marca como destruido.
Jugador	Vidas del jugador; se descuentan cuando un enemigo llega al final del camino.
Oleada	Plantilla de una oleada: cantidad de enemigos, tipo, vida base y velocidad base.
NodoEnemigo	Nodo de la lista doblemente enlazada: referencia al Enemigo y a los nodos anterior/siguiente.
NodoOleada	Nodo de la lista circular: referencia a la Oleada y al nodo siguiente.
ListaSecuencialTorres	Arreglo Torre[] + contador cantidad; inserta, busca, elimina y recorre las torres registradas.
ListaDobleEnemigos	Lista doblemente enlazada con primero/ultimo; inserta, elimina y recorre los enemigos activos en ambos sentidos.
ListaCircularOleadas	Lista circular simple con referencia a ultimo; registra oleadas y avanza el ciclo indefinidamente.
TowerDefenseApp	Aplicación de consola (entregable obligatorio): contiene las tres estructuras, el estado de la partida, el menú y el método main.
Main	Punto de entrada alterno que únicamente invoca a TowerDefenseApp.main().
TableroPanel	Panel Swing (JPanel) que dibuja el tablero de la interfaz gráfica opcional: camino, torres, enemigos y su vida, y las animaciones de disparo.
TowerDefenseGUI	Programa alterno con interfaz gráfica (Swing) sobre las mismas tres estructuras y las mismas clases de datos que TowerDefenseApp, pero con tablero animado en lugar de menú de consola; no se ejecuta junto con TowerDefenseApp.

3. ¿Qué son las estructuras utilizadas y por qué se aplican aquí?

Lista secuencial (arreglo + contador): ListaSecuencialTorres guarda las torres en un arreglo Torre[] datos de capacidad fija, con un contador cantidad que marca las posiciones ocupadas (siempre datos[0..cantidad-1]). Se eligió porque el número de torres es siempre pequeño y se recorre por completo en cada turno para verificar el rango de ataque, donde el costo de un arreglo compacto es despreciable frente a su simplicidad.

Lista doblemente enlazada: ListaDobleEnemigos guarda cada enemigo activo en un NodoEnemigo con referencias a anterior y siguiente, manteniendo primero y ultimo. Se eligió porque los enemigos se agregan y eliminan constantemente en cada turno; un arreglo obligaría a desplazar elementos en cada baja, mientras que el doble enlace elimina un nodo ya localizado en O(1) y permite el recorrido en ambos sentidos que exige el enunciado.

Lista circular simple: ListaCircularOleadas encadena los NodoOleada de modo que el siguiente del último apunta siempre al primero. Se eligió porque las oleadas se repiten en ciclo por diseño del juego, y la circularidad evita programar aparte la lógica de “volver al inicio” cuando se termina la última oleada registrada.

4. Relación entre requisitos y código

Requisito	Aplicación en el código
-----------	-------------------------



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



Lista secuencial de torres	ListaSecuencialTorres (arreglo Torre[] + cantidad), sin colecciones de java.util.
Lista doblemente enlazada de enemigos	ListaDobleEnemigos + NodoEnemigo (referencias anterior/siguiente, primero/ultimo).
Lista circular de oleadas	ListaCircularOleadas + NodoOleada (el siguiente del último apunta al primero).
Inserción y eliminación de torres	insertarTorre() al final en O(1); eliminarTorrePorId() localiza y desplaza el arreglo en O(n).
Eliminación de enemigos sin desplazar elementos	eliminarEnemigoDestruído() reenlaza anterior.siguiente y siguiente.anterior en O(1) una vez localizado el nodo.
Recorrido bidireccional de enemigos	recorrerHaciaAdelante() desde primero y recorrerHaciaAtras() desde ultimo.
Ciclo continuo de oleadas	avanzarSiguienteOleada() e reiniciarCiclo() sobre la lista circular.
Restricción de no usar java.util	Ninguna clase importa ArrayList, LinkedList, Queue ni Deque; todo se construye con arreglos y referencias entre objetos.
Encapsulamiento	Atributos private en Torre, Enemigo, Jugador, Oleada y en las listas, con acceso mediante métodos públicos.
Entregable con main	TowerDefenseApp.main(); Main solo lo invoca como punto de entrada alterno.

5. Paso 1: Diseñar las clases de datos base

```
public class Torre {
    private int id, posicion, danio, rango, costo;
    private String nombre, tipo;
    public boolean enRango(int posicionEnemigo) {
        return Math.abs(posicionEnemigo - posicion) <= rango;
    }
}

public class Enemigo {
    private int id, vida, velocidad, posicion, recompensa;
    private String tipo;
    public void avanzar() { posicion += velocidad; }
    public void recibirDanio(int danio) { vida -= danio; if (vida < 0) vida = 0; }
    public boolean estaDestruído() { return vida <= 0; }
}
```

Torre y Enemigo concentran los atributos y el comportamiento propio de cada entidad (rango de ataque, avance y recepción de daño), manteniendo el estado encapsulado con atributos private y métodos de acceso.

6. Paso 2: Implementar la lista secuencial de torres

```
public class ListaSecuencialTorres {
    private Torre[] datos;
    private int cantidad;

    public boolean insertarTorre(Torre t) {
        if (estaLlena() || buscarIndice(t.getId()) != -1) return false;
        datos[cantidad] = t;
        cantidad++;
        return true;
    }

    public boolean eliminarTorrePorId(int id) {
        int pos = buscarIndice(id);
```



```
        if (pos == -1) return false;
        for (int i = pos; i < cantidad - 1; i++) datos[i] = datos[i + 1];
        datos[cantidad - 1] = null;
        cantidad--;
        return true;
    }
}
```

Las posiciones válidas del arreglo son siempre datos[0..cantidad-1]. insertarTorre() rechaza la operación si la lista está llena o si el id ya existe; eliminarTorrePorId() desplaza los elementos posteriores una posición a la izquierda para no dejar huecos.

7. Paso 3: Implementar los nodos y la lista doblemente enlazada de enemigos

```
public class NodoEnemigo {
    private Enemigo enemigo;
    private NodoEnemigo anterior, siguiente;
}

public class ListaDobleEnemigos {
    private NodoEnemigo primero, ultimo;
    private int tamano;

    public boolean eliminarEnemigoDestruido(int id) {
        NodoEnemigo nodo = buscarEnemigoPorId(id);
        if (nodo == null) return false;
        NodoEnemigo ant = nodo.getAnterior();
        NodoEnemigo sig = nodo.getSiguiente();
        if (ant != null) ant.setSiguiente(sig); else primero = sig;
        if (sig != null) sig.setAnterior(ant); else ultimo = ant;
        tamano--;
        return true;
    }
}
```

Al eliminar un enemigo, la lista reenlaza directamente los nodos vecinos sin desplazar ningún otro elemento del camino, lo que evita el costo $O(n)$ que tendría un arreglo cuando los enemigos se destruyen constantemente.

8. Paso 4: Implementar los nodos y la lista circular de oleadas

```
public class NodoOleada {
    private Oleada oleada;
    private NodoOleada siguiente;
}

public class ListaCircularOleadas {
    private NodoOleada ultimo, actual;

    public void registrarOleada(Oleada o) {
        NodoOleada nuevo = new NodoOleada(o);
        if (estaVacia()) {
            nuevo.setSiguiente(nuevo);
            ultimo = nuevo;
        } else {
            nuevo.setSiguiente(ultimo.getSiguiente());
            ultimo.setSiguiente(nuevo);
            ultimo = nuevo;
        }
    }

    public Oleada avanzarSiguienteOleada() {
        if (estaVacia()) return null;
        actual = (actual == null) ? ultimo.getSiguiente() :
        actual.getSiguiente();
        return actual.getOleada();
    }
}
```



```
}
```

La lista solo guarda la referencia ultimo, porque ultimo.getSiguiente() ya es el primero. Al registrar una nueva oleada, el nuevo nodo se inserta justo después de ultimo preservando la circularidad; avanzarSiguienteOleada() mueve el puntero actual una posición en el círculo, y al pasar la última oleada vuelve sola a la primera sin lógica adicional.

9. Paso 5: Registrar y eliminar torres desde el menú

```
Torre t = new Torre(siguienteIdTorre, nombre, tipo, posicion, danio, rango, costo);  
if (torres.insertarTorre(t)) {  
    System.out.println("Torre registrada con id " + t.getId() + ".");  
    siguienteIdTorre++;  
}
```

El identificador de cada torre se genera de forma incremental (siguienteIdTorre) y solo se confirma si insertarTorre() retorna true, evitando ids duplicados en la lista.

10. Paso 6: Registrar oleadas e iniciar la generación de enemigos

```
Oleada o = new Oleada(siguienteIdOleada, cantidad, tipoEnemigo, vidaBase, velocidadBase);  
oleadas.registrarOleada(o);  
  
Oleada actual = oleadas.avanzarSiguienteOleada();  
for (int i = 0; i < actual.getCantidadEnemigos(); i++) {  
    Enemigo e = new Enemigo(siguienteIdEnemigo, actual.getTipoEnemigo(),  
        actual.getVidaBase(), actual.getVelocidadBase(), 0,  
        actual.getVidaBase());  
    enemigos.insertarEnemigoAlFinal(e);  
    siguienteIdEnemigo++;  
}
```

Al iniciar la siguiente oleada, se crean tantos objetos Enemigo como indique cantidadEnemigos y se insertan al final de ListaDobleEnemigos, todos partiendo de la posición 0 del camino.

11. Paso 7: Implementar la lógica de “avanzar turno”

```
enemigos.actualizarPosicionEnCadaTurno(); // 1. mover enemigos  
  
for (Torre t : torres.getTorres()) { // 2. atacar  
    NodoEnemigo actual = enemigos.getPrimero();  
    while (actual != null) {  
        Enemigo e = actual.getEnemigo();  
        if (!e.estaDestruido() && t.enRango(e.getPosicion())) {  
            e.recibirDanio(t.getDanio());  
        }  
        actual = actual.getSiguiente();  
    }  
}  
// 3. eliminar destruidos, 4. descontar vidas por fuga, 5. imprimir resumen
```

El turno se procesa en cinco pasos: mover enemigos según su velocidad, aplicar el daño de cada torre a los enemigos dentro de su rango, eliminar los enemigos cuya vida llegó a 0, descontar una vida al jugador por cada enemigo que alcanzó el final del camino (y retirarlo), e imprimir el resumen del turno.

12. Tipos de datos utilizados

Variables	Tipo / Categoría	Descripción / Uso
id, posicion, danio, rango, costo, vida, velocidad,	int (primitivo)	Identificadores, coordenadas y estadísticas numéricas enteras.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



recompensa, cantidad, tamaño		
estaDestruído(), derrotado(), estaVacía(), estaLlena()	boolean (primitivo)	Resultados lógicos de validación de estado.
nombre, tipo (Torre/Enemigo/Oleada)	String (referencia)	Nombre y categoría descriptiva de torres, enemigos y oleadas.
Torre[] datos	Arreglo de objetos (referencia)	Almacenamiento interno de ListaSecuencialTorres.
NodoEnemigo, NodoOleada	Clases de nodo (referencia)	Enlazan los objetos Enemigo/Oleada dentro de sus listas respectivas.
primero, ultimo, actual, anterior, siguiente	Referencias a nodo	Punteros que sostienen la lista doble y la lista circular.

13. Diagrama de clases

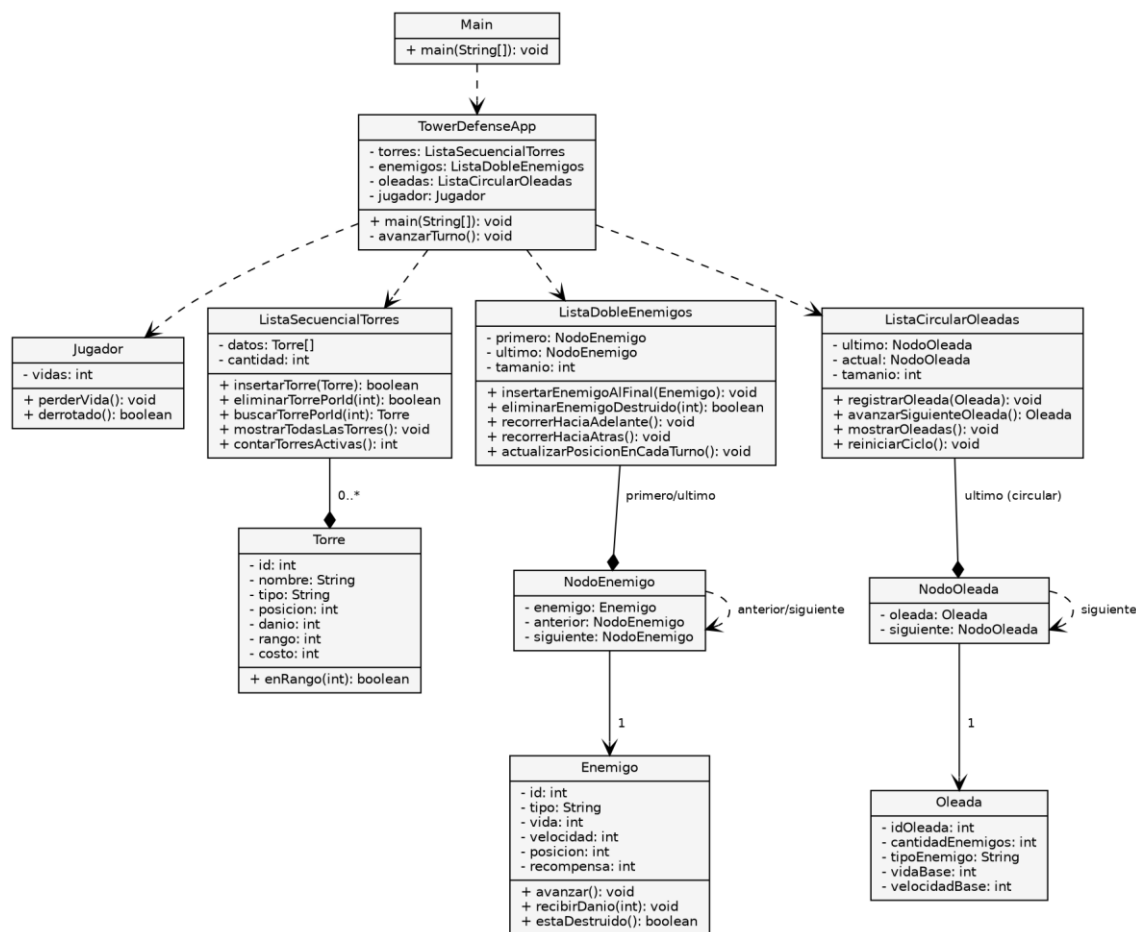


Figura 1. Diagrama de clases del simulador Tower Defense (examen).

14. Especificación de las tres estructuras

Estructura	Estado	Operaciones principales
------------	--------	-------------------------



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



ListaSecuencialTorres	datos: Torre[], cantidad: int	insertarTorre, eliminarTorrePorId, buscarTorrePorId, mostrarTodasLasTorres, contarTorresActivas, getTorres
ListaDobleEnemigos	primero: NodoEnemigo, ultimo: NodoEnemigo, tamano: int	insertarEnemigoAlFinal, buscarEnemigoPorId, eliminarEnemigoDestruido, recorrerHaciaAdelante, recorrerHaciaAtras, actualizarPosicionEnCadaTurno
ListaCircularOleadas	ultimo: NodoOleada, actual: NodoOleada, tamano: int	registrarOleada, avanzarSiguienteOleada, mostrarOleadas, reiniciarCiclo, getOleadasComoArreglo, getOleadaActual

Invariantes:

1. En ListaSecuencialTorres, las posiciones válidas son siempre datos[0 .. cantidad-1]; el resto del arreglo permanece en null.
2. $0 \leq \text{cantidad} \leq \text{datos.length}$ en ListaSecuencialTorres (nunca se inserta si estaLlena() es true).
3. No existen dos torres con el mismo id dentro de ListaSecuencialTorres.
4. En ListaDobleEnemigos, primero.getAnterior() siempre es null y ultimo.getSiguiente() siempre es null mientras la lista no esté vacía.
5. Un Enemigo con vida ≤ 0 se considera destruido y debe retirarse de ListaDobleEnemigos en el mismo turno en que ocurre.
6. En ListaCircularOleadas, ultimo.getSiguiente() siempre apunta al primer nodo registrado (la lista nunca deja de ser circular).
7. El puntero actual de ListaCircularOleadas solo avanza mediante avanzarSiguienteOleada(); reiniciarCiclo() es la única operación que lo regresa a null.
8. El jugador nunca tiene vidas negativas (perderVida() no decrementa por debajo de 0).
9. La partida se declara en curso hasta que el jugador es derrotado o se cumplen todas las oleadas sin enemigos activos.

15. Diagrama de flujo del programa principal

INICIO (Main -> TowerDefenseApp.main)
 crear ListaSecuencialTorres, ListaDobleEnemigos, ListaCircularOleadas,
 Jugador(3 vidas)

REPETIR

 mostrar menu de 10 opciones
 leer opcion

SEGUN opcion HACER

- 1: registrar torre
- 2: mostrar torres registradas
- 3: eliminar torre por id
- 4: registrar oleada
- 5: mostrar oleadas (recorrido circular)
- 6: iniciar siguiente oleada (avanzarSiguienteOleada)
- 7: avanzar turno (mover, atacar, eliminar, descontar vidas)
- 8: mostrar enemigos activos (adelante o atras)
- 9: mostrar estado general del juego
- 10: salir

HASTA QUE opcion = 10 o el jugador salga



FIN

16. Prueba de escritorio

Caso de prueba: Torre Arquero (posición 3, daño 20, rango 2), Torre Cañón (posición 8, daño 35, rango 3), Oleada de 3 enemigos básicos (vida 50, velocidad 1), jugador con 3 vidas. Se registran ambas torres y la oleada, se inicia la oleada y se avanza el turno repetidamente:

Turno	Posición de los enemigos	Vida de cada enemigo básico
1	1	50 (fuera de rango del Arquero: $ 1-3 = 2 \leq 2$, sí en rango)
2	2	30
3	3	10

Resultado: los tres enemigos básicos son destruidos en el turno 3 sin que el jugador pierda ninguna vida, tal como confirma el caso de prueba documentado en el proyecto.

Caso adicional	Operación / Estado previo	Resultado esperado
Lista de torres vacía	torres.mostrarTodasLasTorres() sin torres registradas	Imprime “No hay torres registradas.” y no falla.
Lista de torres llena	insertarTorre() cuando cantidad == datos.length	estaLlena() retorna true; insertarTorre() retorna false y no agrega la torre.
Id de torre repetido	insertarTorre() con un id ya existente en el arreglo	buscarIndice(id) distinto de -1; insertarTorre() retorna false sin duplicar la torre.
Lista de enemigos vacía	enemigos.recorrerHaciaAdelante() / HaciaAtras() sin enemigos activos	Imprime “No hay enemigos activos.” y no falla.
Avanzar turno sin enemigos	avanzarTurno() con ListaDobleEnemigos vacía	Imprime “No hay enemigos activos para avanzar el turno.” y no aplica daño ni vidas.
Oleadas vacías al iniciar siguiente	iniciarSiguienteOleada() sin oleadas registradas	Imprime “No hay oleadas registradas.” y no crea enemigos.
Ciclo circular completo	avanzarSiguienteOleada() llamado más veces que oleadas registradas	El puntero actual vuelve a la primera oleada registrada, confirmando la circularidad de la lista.

2.3 Conclusiones

- La ListaSecuencialTorres garantizó un control estático y ordenado de las defensas manteniendo la integridad de los índices al eliminar elementos.
- La ListaDobleEnemigos optimizó el rendimiento del ciclo de juego al permitir la remoción de unidades destruidas en tiempo constante $O(1)$.
- La ListaCircularOleadas proporcionó un flujo de ataque continuo, dinámico y reutilizable entre cada fase del juego.

2.4 Recomendaciones

Redactar las sugerencias o acciones concretas que se derivan de la guía práctica desarrollada.

- Se sugiere implementar un factor de crecimiento dinámico en ListaSecuencialTorres para superar el límite fijo actual y soportar partidas extensas.



- Conviene separar la lógica de simulación de TowerDefenseGUI hacia un controlador independiente para eliminar la duplicación de código entre la consola y la GUI.
- Se recomienda aplicar el patrón Object Pool en la gestión de animaciones para evitar sobrecargas de memoria por el uso de arreglos fijos.

2.5 Referencias bibliográficas

[1] Baeldung, "Reduce Object Header Size and Save Memory in Java 25," Baeldung, may 2026. Accedido el 30 de agosto de 2026. [En línea]. Disponible: <https://www.baeldung.com/java-object-header-reduced-size-save-memory>

[2] "The Cost of Garbage Collection for State Machine Replication," arXiv preprint, arXiv:2405.11182, 2024. Accedido el 30 de agosto de 2026. [En línea]. Disponible: <https://arxiv.org/pdf/2405.11182>